

Codefest 2 Solutions

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

ECE 410/510

Spring 2026

CMAN

Hardware specification

Peak compute $P_{\text{peak}} = 10 \text{ TFLOPS (FP32)} = 10 \times 10^{12} \text{ FLOP/s}$. Peak DRAM bandwidth $B_{\text{peak}} = 320 \text{ GB/s}$. Ridge point $I^* = P_{\text{peak}} / B_{\text{peak}} = 10,000 / 320 = 31.25 \text{ FLOP/byte}$.

The roofline defines attainable performance as $P_{\text{attainable}}(I) = \min(I \times B_{\text{peak}}, P_{\text{peak}})$. Any kernel with $AI < 31.25 \text{ FLOP/byte}$ is memory-bandwidth-bound; any kernel with $AI > 31.25 \text{ FLOP/byte}$ is compute-bound.

Task 2: Kernel A — Dense GEMM 1024×1024 (FP32)

FLOPs

For a square matmul of size $N \times N$, $\text{FLOPs} = 2N^3$ (one multiply + one accumulate per output element, N^2 output elements, N operations each).

$$\text{FLOPs} = 2 \times 1024^3 = 2,147,483,648 \approx 2.15 \times 10^9 \text{ FLOP}$$

Bytes transferred (no cache reuse)

Three matrices A, B, C each of size 1024×1024 FP32 elements (4 bytes each) are loaded or stored from DRAM with no reuse assumed:

$$\text{Bytes} = 3 \times 1024^2 \times 4 = 12,582,912 \text{ bytes} \approx 12.58 \text{ MB}$$

Arithmetic intensity

$$AI = 2,147,483,648 / 12,582,912 \approx 170.7 \text{ FLOP/byte}$$

Bound classification and attainable performance

$AI = 170.7 \text{ FLOP/byte} > I^* = 31.25 \text{ FLOP/byte}$, so GEMM is **compute-bound** on this hardware.

Attainable performance = $P_{\text{peak}} = 10,000 \text{ GFLOP/s}$.

Note: The S/NY criteria state “memory-bound classification correct” for GEMM. This is an error in the criteria document. With the specified hardware ($P_{\text{peak}} = 10 \text{ TFLOPS}$, $B_{\text{peak}} = 320 \text{ GB/s}$, $I^* = 31.25 \text{ FLOP/byte}$), GEMM has $AI = 170.7 \text{ FLOP/byte}$ and is compute-bound. Accept either classification if a student shows the correct AI value and compares it to the ridge point. Do not penalize a student who correctly identifies GEMM as compute-bound.

Architectural recommendation

GEMM is compute-bound, so the bottleneck is arithmetic throughput, not memory bandwidth. The most effective architectural change is to **increase peak compute throughput** (e.g., wider SIMD units, more FMA lanes, or a dedicated matrix-multiply accelerator such as a tensor core). Adding more memory bandwidth would not improve performance because the kernel is already compute-limited.

Task 3: Kernel B — Vector addition, $N = 4,194,304$ (FP32)

FLOPs

One addition per element:

$$\text{FLOPs} = 4,194,304 \approx 4.19 \times 10^6 \text{ FLOP}$$

Bytes transferred (no cache reuse)

Two input vectors (A, B) are read and one output vector (C) is written, each of N FP32 elements (4 bytes):

$$\text{Bytes} = 3 \times 4,194,304 \times 4 = 50,331,648 \text{ bytes} \approx 50.3 \text{ MB}$$

Arithmetic intensity

$$\text{AI} = 4,194,304 / 50,331,648 = 1/12 \approx 0.083 \text{ FLOP/byte}$$

Bound classification and attainable performance

AI = 0.083 FLOP/byte $\ll I^* = 31.25 \text{ FLOP/byte}$, so vector-add is **memory-bandwidth-bound**. Attainable performance = AI $\times$ B_{peak} = 0.083 $\times$ 320 $\approx 26.7 \text{ GFLOP/s}$, far below P_{peak} = 10,000 GFLOP/s. The compute units are almost entirely idle.

Architectural recommendation

Vector-add is deeply memory-bound with AI = 0.083 FLOP/byte. Adding more compute units would not help. The most effective changes are to **increase memory bandwidth** (e.g., use HBM instead of GDDR, or widen the memory bus), or to **fuse vector-add with adjacent operations** so that data loaded once from DRAM is reused for multiple computations, raising the effective arithmetic intensity.

Summary table

Kernel	FLOPs	Bytes	AI (FLOP/byte)	Bound	Attainable
GEMM 1024×1024	2.15×10^9	12.58 MB	170.7	Compute	10,000 GFLOP/s
Vector-add N=4,194,304	4.19×10^6	50.3 MB	0.083	Memory	26.7 GFLOP/s

Ridge point $I^* = 31.25 \text{ FLOP/byte}$. A kernel with AI above 31.25 is compute-bound; below 31.25 is memory-bound.

CLLM

The CLLM tasks are project-specific and cannot have a single correct answer. Apply the following principles when grading:

- **Profiler output (project_profile.txt)**. Accept any cProfile, line_profiler, or torch.profiler output that shows per-function or per-layer timing. The dominant kernel must be explicitly named with a percentage of total runtime.
- **Arithmetic intensity (ai_calculation.md)**. The FLOPs formula must be shown analytically with variable names substituted. The bytes estimate must separate operand types (weights, inputs, outputs). The final AI value must be stated in FLOP/byte. Accept any value consistent with the student's stated kernel and derivation; no single expected answer exists.
- **Roofline plot (roofline_project.png)**. Must show axes labeled (x: FLOP/byte log scale, y: GFLOP/s log scale), ridge point labeled, the student's software kernel plotted at their computed AI, and a separate hypothetical HW accelerator design point. The HW design point does not need to be optimal; it must be labeled and placed at a plausible position above the software kernel.
- **Partition rationale (partition_rationale.md)**. Must be at least 200 words and address all four sub-questions: (a) which kernel to accelerate and roofline justification, (b) what stays in software, (c) required interface bandwidth in GB/s with calculation, (d) bound classification and whether the HW design changes it. A complete answer to (c) must show: throughput target $\times$ data width = required GB/s, compared to a named interface's rated bandwidth.
- **Heilmeier questions (project/heilmeier.md)**. Q2 must reference a specific runtime measurement or profiling result. "Current software is slow" is not acceptable. Q3 must reference the arithmetic intensity

result and/or roofline position. A vague answer to Q3 that does not connect to the profiling data is Not Yet.

Sample CLLM calculations (CNN backprop reference)

For students using the course's CNN backpropagation sample algorithm, the dominant kernel is Conv2D._im2col. The FLOPs formula for one Conv2D forward pass is:

$$\text{FLOPs} = 2 \times N \times C_{\text{in}} \times k^2 \times H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}$$

Bytes transferred (FP64, no reuse): input patch ($N \times C_{\text{in}} \times k^2 \times H_{\text{out}} \times W_{\text{out}}$) + weights ($C_{\text{out}} \times C_{\text{in}} \times k^2$) + output ($N \times C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}$), each multiplied by 8 bytes (FP64).

For the tabular Q-learning sample: dominant kernel is QLearningAgent.update; FLOPs $\approx$ 13 per step; bytes = 72 per step (read $Q[s,:]$ and $Q[s',:]$, write one element); AI $\approx$ 0.18 FLOP/byte, deeply memory-bound. Hardware acceleration of tabular Q-learning is limited by random Q-table lookups, not arithmetic throughput.